

# คู่มือ Database Migration

สำหรับระบบ Tarawit · Dev → Production

หลักสำคัญ: เปลี่ยนโครงสร้างฐานข้อมูลทุกครั้ง ต้องสร้าง migration ใหม่

ฉบับใช้งานจริง · กันข้อมูลหาย · ทำตามได้ที่ละขั้น

# ภาพรวม: Migration คืออะไร?

Migration คือไฟล์ SQL ที่บอกฐานข้อมูลว่าโครงสร้างต้องเปลี่ยนจากเวอร์ชันเดิมไปเป็นเวอร์ชันใหม่อย่างไร โดยระบบจะบันทึกว่าไฟล์ใดทำงานแล้ว จึงไม่รันซ้ำโดยไม่ตั้งใจ.



## กฎทอง

ไฟล์ที่ apply แล้วถือเป็นประวัติ ห้ามย้อนกลับไปแก้ไข  
ให้สร้างไฟล์เลขถัดไปเพื่อแก้ไขแทน

## โครงสร้างในโปรเจกต์

```
server/db/migrations/  
000011_add_nickname.sql  
000012_add_department_id.sql  
000013_add_evaluation_assignment_comment.sql  
000014_your_next_change.sql
```

## ใครทำหน้าที่อะไร

| ส่วน              | หน้าที่                                     |
|-------------------|---------------------------------------------|
| ไฟล์ .sql         | เปลี่ยน schema หรือย้ายข้อมูล               |
| schema_migrations | บันทึกว่า migration ใด apply แล้ว           |
| migrate status    | แสดง applied และ pending โดยไม่แก้ DB       |
| migrate up        | apply migration ที่ pending ทั้งหมดตามลำดับ |

# 1 · สร้าง Migration ใหม่

1

## หาเลขล่าสุด

ดูชื่อไฟล์ทั้งหมดและเรียงตามเลข

```
ls server/db/migrations/*.sql | sort
```

2

## ตั้งชื่อไฟล์

ใช้เลข 6 หลักต่อจากไฟล์ล่าสุด ตามด้วยคำอธิบายภาษาอังกฤษแบบ snake\_case

```
000014_add_evaluation_deadline.sql
```

3

## เขียน SQL แบบปลอดภัย

เพิ่มคอลัมน์ก่อนและใช้ IF NOT EXISTS เมื่อคำสั่งรองรับ

```
ALTER TABLE evaluation_instances
ADD COLUMN IF NOT EXISTS deadline TIMESTAMPTZ;

CREATE INDEX IF NOT EXISTS idx_evaluation_deadline
ON evaluation_instances(deadline);
```

### ควรทำ

เพิ่มคอลัมน์เป็น nullable หรือกำหนด DEFAULT เพื่อให้ข้อมูลเดิมยังใช้งานได้

### หลีกเลี่ยง

DROP COLUMN, เปลี่ยนชนิดข้อมูลทันที หรือเพิ่ม NOT NULL โดยยังไม่เติมข้อมูลเดิม

## รูปแบบ 3 ขั้นตอนสำหรับการเปลี่ยนใหญ่

รอบ 1: เพิ่มของใหม่โดยยังรองรับของเก่า → รอบ 2: ย้ายข้อมูลและเปลี่ยนโค้ด → รอบ 3: ลบของเก่าหลังตรวจว่าไม่มีผู้ใช้งานแล้ว

## 2 · ตรวจสอบและ Apply ใน Dev

1

### ตรวจสอบสถานะก่อนทุกครั้ง

คำสั่งนี้อ่านอย่างเดียว ไม่เปลี่ยนฐานข้อมูล

```
docker compose -f docker-compose.dev.yml exec -T server \  
go run ./cmd/migrate status
```

2

### อ่านรายการ pending

ตรวจสอบให้แน่ใจว่าไม่มี migration แปลกปลอมหรือยังไม่ได้ review

3

### Apply migration

คำสั่ง up จะรัน pending ทั้งหมด ไม่ได้เลือกเฉพาะไฟล์ล่าสุด

```
docker compose -f docker-compose.dev.yml exec -T \  
-e MIGRATION_CONFIRM_DB=tarawitDB \  
server go run ./cmd/migrate up
```

4

### ตรวจสอบ schema จริง

เข้า psql แล้วตรวจสอบตารางและประวัติ migration

```
docker compose -f docker-compose.dev.yml exec db \  
psql -U yeawyow -d tarawitDB  
  
\d evaluation_instances  
SELECT name, applied_at FROM schema_migrations ORDER BY name;  
\q
```

จำไว้

status ก่อน up เสมอ เพราะ up จะ apply ทุกไฟล์ที่ pending ตามลำดับ

### 3 · ถ้าแก้พลาด ทำอย่างไร?

ให้ถามก่อนว่า migration ถูก apply ไปแล้วหรือยัง

| ยังไม่ Apply                                                        | Apply แล้ว                                          |
|---------------------------------------------------------------------|-----------------------------------------------------|
| แก้ไฟล์เดิมได้<br>เพราะยังไม่มีฐานข้อมูลใดบันทึกว่าไฟล์นี้ทำงานแล้ว | ห้ามแก้ไฟล์เดิม ให้สร้าง migration เลขถัดไปเพื่อแก้ |
| <pre>000014_...sql status</pre>                                     | <pre>000015_fix_deadline_type.sql</pre>             |

### ตัวอย่างแก้ไขข้อมูลในไฟล์ใหม่

```
ALTER TABLE evaluation_instances
ALTER COLUMN deadline TYPE TIMESTAMPTZ
USING deadline AT TIME ZONE 'Asia/Bangkok';
```

#### ห้ามทำ

อย่าลบแถวออกจาก schema\_migrations เพื่อบังคับรันซ้ำ เพราะ SQL อาจทำงานซ้ำและทำให้ข้อมูลเสียหาย

#### ก่อนทดลอง

ถ้า migration แต่ละข้อมูลจำนวนมาก ให้สร้าง backup ของ Dev ก่อน แม้จะไม่ใช้ Production

## 4 · นำขึ้น Production อย่างปลอดภัย

Production ต้องแยกการเปลี่ยน schema ออกจากการตัดสินใจ restore ข้อมูล และต้องมี backup ก่อนทุกครั้ง

1

### Commit ให้ครบชุด

migration, API model/query และ UI ต้องอยู่ใน revision เดียวกัน

2

### สร้าง ARM64 binary

API บน Pi ใช้ binary ที่ build ล่วงหน้า

```
make -C server build
```

3

### ตรวจและ Backup

deploy.sh จะรัน pre-deploy check และสร้าง logical backup

4

### Deploy

สคริปต์จะแสดง migration status, apply pending migrations แล้ว replace API

```
./deploy.sh
```

5

### ตรวจหลัง Deploy

ดู health, logs, schema และทดสอบ flow ที่เกี่ยวข้อง

#### ลำดับสำคัญ

ออกแบบให้ API ช่วงเปลี่ยนผ่านรองรับ schema เดิมและใหม่ได้ ถ้า migration กับการเปลี่ยน container ไม่เกิดพร้อมกันพอดี

## Rollback

โดยทั่วไปให้ rollback เฉพาะ API ก่อน ถ้า migration เป็นแบบ additive เช่นเพิ่ม nullable column API เก่ายังทำงานได้ การ restore DB ควรเป็นทางเลือกสุดท้าย เพราะข้อมูลใหม่หลังเวลา backup จะหาย

# Checklist ใช้งานจริง

|   |                 |                                                                                |
|---|-----------------|--------------------------------------------------------------------------------|
| □ | ก่อนเขียน       | เลข migration ไม่ซ้ำ · ชื่อบอกรหัสประสงค์ · หนึ่งไฟล์หนึ่งเรื่อง               |
| □ | ก่อน Apply Dev  | รัน status · อ่าน pending ทุกไฟล์ · ตรวจสอบ SQL ว่าไม่ลบข้อมูล                 |
| □ | หลัง Apply Dev  | ตรวจสอบ schema · ทดสอบ API/UI · ดู log · รัน tests/build                       |
| □ | ก่อน Commit     | migration อยู่พร้อมโค้ดที่ใช้งาน schema ใหม่ · ไม่ commit secrets หรือ dump    |
| □ | ก่อน Production | backup · pre-deploy check · ตรวจสอบพื้นที่ disk · ตรวจสอบ binary ARM64         |
| □ | หลัง Production | health ผ่าน · migration applied · login/บันทึก/อ่านข้อมูลได้ · ไม่มี error log |

## สูตรง่าย

|          |                                                                                            |
|----------|--------------------------------------------------------------------------------------------|
| ทุกครั้ง | สร้างไฟล์ใหม่ → status → up ใน Dev → ทดสอบ → commit พร้อมโค้ด → backup → deploy Production |
|----------|--------------------------------------------------------------------------------------------|

## คำสั่งสั้นที่ใช้บ่อย

```
# ■■■■■■■■■■
ls server/db/migrations/*.sql | sort

# ■■■■■■■■■■
docker compose -f docker-compose.dev.yml exec -T server go run ./cmd/migrate status

# Apply Dev
docker compose -f docker-compose.dev.yml exec -T -e MIGRATION_CONFIRM_DB=tarawitDB server go run
./cmd/migrate up
```